Python Fundamentals and Applications: A Comprehensive Guide

Ronald Parent

Los Angeles City College

Python Fundamentals and Applications: A Comprehensive Guide

Chapter 1: Introduction to Python

1.1 What is Python?

Python stands as an interpreted, object-oriented, high-level programming language characterized by dynamic semantics. Its design philosophy emphasizes code readability, employing significant indentation. Key attributes include high-level built-in data structures combined with dynamic typing and dynamic binding, which collectively make it exceptionally attractive for Rapid Application Development (RAD). Furthermore, Python serves effectively as a scripting or "glue" language, adept at interconnecting existing software components.[1]

Programmers often report increased productivity when using Python. This stems largely from its interpreted nature, which eliminates a separate compilation step, thereby accelerating the edit-test-debug cycle. Debugging is further aided by the interpreter's behavior of raising exceptions upon encountering errors, typically accompanied by a stack trace. Source-level debuggers are also readily available to assist in pinpointing issues.[1]

Python's syntax is notably simple and straightforward, contributing to its ease of learning and reducing the long-term costs associated with program maintenance. The language inherently supports modularity through modules and packages, promoting code reuse and organized project structures.[1]

1.2 Setting Up Python

The Python interpreter, along with its extensive standard library, is freely available in both source and binary formats for all major operating systems and can be distributed without charge.[1]

Installation procedures vary slightly by operating system:

1. **Download:** Obtain the appropriate installer from the official Python website (https://www.python.org/downloads/).[1] The site typically highlights the latest stable release for Windows and macOS, while providing source code for Linux/Unix systems.
2. **Run Installer (Windows/macOS):** Execute the downloaded installer (.exe for Windows, .pkg for macOS).

- On Windows, it is highly recommended to select the option "Add Python X.Y to PATH" during installation to ensure Python can be easily run from the command line.[1]
- On macOS, follow the installer prompts.

3. **Compile Source (Linux/Unix):** For most Unix-like systems, installation involves downloading the source tarball and compiling it using standard commands like ./configure, make, and sudo make install in the terminal.[1] Specific commands may vary by distribution.
4. **Verification:** After installation, open a terminal or command prompt and execute python --version or python3 --version. A successful installation will display the installed Python version number.[1]

1.3 Basic Syntax, Variables, and Data Types

Python's syntax prioritizes readability. A fundamental aspect distinguishing Python from many other languages is its use of indentation, rather than braces or keywords, to define code blocks (e.g., within loops, functions, or conditional statements).[2] This enforces a consistent visual structure but requires careful attention, as incorrect indentation leads to IndentationError.

A simple "Hello, World!" program illustrates basic syntax and the print() function:

Python

```
# This is a single-line comment
print("Hello, World!")
```

Variable Assignment: Variables are created when a value is assigned to them using the equals sign (=). Python uses dynamic typing, meaning you don't need to declare the variable's type explicitly; the type is inferred from the assigned value.[3]

Python

```
message = "This is a string"  # String assignment
count = 100                   # Integer assignment
```

```
pi_approx = 3.14159        # Float assignment
is_active = True           # Boolean assignment

print(message)
print(count)
print(pi_approx)
print(is_active)
```

Data Types: Python includes several built-in data types [3]:

- **Integers (int):** Whole numbers (e.g., -5, 0, 42).
- **Floating-Point Numbers (float):** Numbers with a decimal point (e.g., 3.14, -0.001, 2.0).
- **Strings (str):** Ordered sequences of characters, enclosed in single (') or double (") quotes (e.g., 'hello', "Python").
- **Booleans (bool):** Represent truth values, True or False.

Comments: Comments explain code and are ignored by the interpreter. Single-line comments start with #. Multi-line comments or docstrings are enclosed in triple quotes ("""Docstring""" or '''Docstring''').[3]

Python

```
# Assigning an integer
counter = 5

"""
This is a multi-line comment
explaining the next section of code.
It uses triple double quotes.
"""
is_complete = False # Assigning a boolean

'''
This is also a multi-line comment,
using triple single quotes.
'''
user_name = "Guido" # Assigning a string
```

1.4 Basic Arithmetic Operations

Python supports standard arithmetic operations.[3]

Python

```
a = 15
b = 4

# Addition
addition = a + b
print(f"{a} + {b} = {addition}") # Output: 15 + 4 = 19

# Subtraction
subtraction = a - b
print(f"{a} - {b} = {subtraction}") # Output: 15 - 4 = 11

# Multiplication
multiplication = a * b
print(f"{a} * {b} = {multiplication}") # Output: 15 * 4 = 60

# Float Division (/)
# Always results in a float, even if the result is a whole number.
division = a / b
print(f"{a} / {b} = {division}") # Output: 15 / 4 = 3.75
print(f"10 / 5 = {10 / 5}")      # Output: 10 / 5 = 2.0

# Integer (Floor) Division (//)
# Divides and rounds down to the nearest whole number (integer).
floor_division = a // b
print(f"{a} // {b} = {floor_division}") # Output: 15 // 4 = 3
print(f"10 // 5 = {10 // 5}")          # Output: 10 // 5 = 2
print(f"-15 // 4 = {-15 // 4}")        # Output: -15 // 4 = -4 (rounds down)

# Modulo (%)
# Returns the remainder of the division.
remainder = a % b
print(f"{a} % {b} = {remainder}") # Output: 15 % 4 = 3
```

```
# Exponentiation (**)
# Raises the first number to the power of the second.
exponentiation = a ** b
print(f"{a} ** {b} = {exponentiation}") # Output: 15 ** 4 = 50625
```

A crucial distinction exists between the / operator, which performs float division, and the // operator, which performs integer (floor) division.[3] In Python 3, / always returns a float, while // discards the fractional part (rounding down towards negative infinity). Misunderstanding this difference can lead to subtle bugs, particularly when integer results are expected or when porting code from languages where / might perform integer division based on operand types.

Chapter 2: Control Flow

Control flow statements direct the order in which code is executed based on conditions or through iteration.

2.1 Conditional Statements: if, elif, else

Conditional statements allow programs to execute different blocks of code based on whether certain conditions evaluate to True or False. The primary structure involves if, optional elif (else if), and an optional final else.[6]

Syntax:

Python

```
if condition1:
    # Code block to execute if condition1 is True
    pass # pass is a null operation - nothing happens
elif condition2:
    # Code block to execute if condition1 is False and condition2 is True
    pass
else:
    # Code block to execute if all preceding conditions are False
    pass
```

Example: Classifying a number.

```Python
try:
    num = float(input("Enter a number: "))

    if num > 0:
        print("The number is positive.")
    elif num == 0:
        print("The number is zero.")
    else: # num must be < 0
        print("The number is negative.")

    # Check for even or odd (only if it's an integer)
    if num == int(num): # Check if it's effectively an integer
        if int(num) % 2 == 0:
            print("It is also an even integer.")
        else:
            print("It is also an odd integer.")
    else:
        print("It is not an integer.")

except ValueError:
    print("Invalid input. Please enter a numeric value.")
```

This if-elif-else structure serves as Python's mechanism for multi-way branching, effectively replacing the switch or case statements found in languages like C++ or Java. This promotes a consistent and arguably simpler syntax for handling conditional logic.[6]

2.2 for Loops

for loops are used to iterate over the items of any sequence (like a list, tuple, string) or other iterable object in the order they appear.[6] Python's for loop functions fundamentally as a "for-each" loop, directly accessing each item without needing explicit index management, which enhances readability and reduces the likelihood of

off-by-one errors common in C-style index-based loops.[6]

Iterating over Sequences:

Python

```
# Iterating over a list
fruits = ["apple", "banana", "cherry"]
print("Fruits:")
for fruit in fruits:
    print(f"- {fruit}")

# Iterating over a string
message = "Python"
print("\nCharacters in 'Python':")
for char in message:
    print(f"- {char}")

# Iterating over a tuple
coordinates = (10, 20, 30)
print("\nCoordinates:")
for coord in coordinates:
    print(f"- {coord}")
```

Using range(): When iteration based on a numerical sequence or a specific number of times is required, the range() function is used. It generates a sequence of numbers.[6]

Python

```
# range(stop): 0 up to (but not including) stop
print("\nNumbers 0 to 4:")
for i in range(5):
    print(i, end=" ") # Output: 0 1 2 3 4
print()
```

```
# range(start, stop): start up to (but not including) stop
print("\nNumbers 2 to 5:")
for i in range(2, 6):
    print(i, end=" ") # Output: 2 3 4 5
print()

# range(start, stop, step): start up to stop, incrementing by step
print("\nEven numbers 0 to 8:")
for i in range(0, 10, 2):
    print(i, end=" ") # Output: 0 2 4 6 8
print()
```

Iterating with Index using enumerate(): To access both the index and the item during iteration, the enumerate() function is convenient.[8]

Python

```
colors = ["red", "green", "blue"]
print("\nColors with index:")
for index, color in enumerate(colors):
    print(f"Index {index}: {color}")

print("\nColors with starting index 1:")
for index, color in enumerate(colors, start=1):
    print(f"Position {index}: {color}")
```

2.3 while Loops

while loops execute a block of code repeatedly as long as a specified boolean condition remains True.[6] It's crucial to ensure that the condition eventually becomes False within the loop's body to prevent infinite loops, unless an infinite loop is intentionally desired (e.g., for a server process).[11]

Syntax:

Python

```
while condition:
    # Code block to execute while condition is True
    # Ensure the condition eventually becomes False (or use break)
    pass
```

Example: Countdown.

Python

```
count = 5
print("Countdown:")
while count > 0:
    print(count)
    count -= 1 # Modify the variable used in the condition
print("Blast off!")
```

Simulating do-while: Python does not have a specific do-while loop construct (which guarantees the loop body executes at least once before checking the condition). However, this behavior can be effectively simulated using a while True loop combined with an if condition and a break statement inside the loop body.[15]

Python

```
# Example: Get positive number input (runs at least once)
while True:
    try:
        user_input = input("Enter a positive number: ")
        number = float(user_input)
        if number > 0:
            print(f"You entered: {number}")
            break # Exit the loop if input is valid
```

```
        else:
            print("Number must be positive.")
    except ValueError:
        print("Invalid input. Please enter a number.")
```

This while True:... if condition: break pattern leverages existing constructs, avoiding the need for additional syntax while providing the required "execute-at-least-once" functionality.[16]

2.4 Loop Control: break and continue

These statements alter the normal flow of loop execution.[6]

- **break:** Immediately terminates the innermost for or while loop it is contained within. Execution resumes at the statement following the terminated loop.[6]
- **continue:** Skips the rest of the code inside the current iteration of the innermost loop and proceeds to the next iteration.[6]

It is important to note that break and continue only affect the loop they are directly inside. When dealing with nested loops, they do not directly control outer loops; alternative logic like setting flags or raising custom exceptions would be needed to break out of multiple loop levels simultaneously.[6]

Example: Using break and continue.

Python

```
print("\nLoop with break and continue:")
for i in range(1, 11): # Numbers 1 to 10
    if i % 2!= 0:
        continue # Skip odd numbers
    if i > 8:
        break # Exit loop if number is greater than 8
    print(i, end=" ") # Print even numbers up to 8: 2 4 6 8
print("\nLoop finished.")

# Example in a while loop
print("\nWhile loop searching for 'stop':")
command = ""
```

```
while True: # Potentially infinite loop
    command = input("Enter command ('stop' to exit, 'skip' to ignore): ")
    if command == "stop":
        print("Exiting loop.")
        break
    if command == "skip":
        print("Skipping this command.")
        continue
    print(f"Processing command: {command}")
```

2.5 Loop else Clause

Python offers a unique feature where for and while loops can have an associated else block. This else block executes *only* if the loop completes its iterations normally, meaning it was *not* terminated prematurely by a break statement.[6] It does execute if the loop condition starts as false or the iterable is empty.

This construct provides a cleaner alternative to using manual flag variables to check if a loop finished because an item was found (triggering break) or because the entire iterable was processed without finding the item.[21] The name else can be slightly counter-intuitive; thinking of it as "no break" often clarifies its purpose.[21]

Example: Searching for an item.

Python

```
items = ["apple", "banana", "orange"]
search_item = "grape"

print(f"\nSearching for '{search_item}'...")
for item in items:
    if item == search_item:
        print(f"Found '{search_item}'!")
        break
else: # Executes only if the loop completes without break
    print(f"'{search_item}' not found in the list.")

search_item = "banana"
```

```
print(f"\nSearching for '{search_item}'...")
for item in items:
    if item == search_item:
        print(f"Found '{search_item}'!")
        break
else:
    print(f"'{search_item}' not found in the list.")
```

Example: Prime number check (from [6]).

Python

```
num_to_check = 29 # Try with 28 as well

print(f"\nChecking if {num_to_check} is prime:")
if num_to_check > 1:
    for i in range(2, num_to_check):
        if (num_to_check % i) == 0:
            print(f"{num_to_check} is not a prime number ({i} is a factor)")
            break
    else: # Executes if the loop finishes without finding a factor
        print(f"{num_to_check} is a prime number")
else:
    print(f"{num_to_check} is not a prime number")
```

2.6 Nested Loops

Loops can be placed inside other loops. The inner loop completes all its iterations for each single iteration of the outer loop.[13] Nested loops are commonly used for tasks like processing multi-dimensional data structures (e.g., matrices), generating combinations or pairs, and creating patterns.[13]

Example: Generating coordinate pairs.

Python

```
print("\nGenerating coordinate pairs (0,0) to (1,2):")
for x in range(2): # Outer loop (x = 0, 1)
    for y in range(3): # Inner loop (y = 0, 1, 2 for each x)
        print(f"({x}, {y})", end=" ")
    print() # Newline after each outer loop iteration
# Output:
# (0, 0) (0, 1) (0, 2)
# (1, 0) (1, 1) (1, 2)
```

Example: Processing a 2D list (matrix).

Python

```
matrix = [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]

print("\nElements in the matrix:")
total_sum = 0
for row in matrix: # Outer loop iterates through rows
    for element in row: # Inner loop iterates through elements in the current row
        print(element, end=" ")
        total_sum += element
    print() # Newline after processing each row
print(f"Sum of all elements: {total_sum}")
```

While syntactically straightforward, nested loops can significantly impact performance, especially with large datasets. The time complexity often increases multiplicatively; for instance, two nested loops iterating N times each typically result in O(N2) complexity.[13] For certain tasks, particularly list creation or data transformation, list comprehensions can offer a more concise and sometimes more efficient alternative to explicit nested loops.[13]

Example: Nested loop vs. List Comprehension for flattening a list.

Python

```
matrix = [[1, 2], [3, 4], [5, 6]]
flattened_loop =
for row in matrix:
    for item in row:
        flattened_loop.append(item)

flattened_comp = [item for row in matrix for item in row]

print(f"\nFlattened (loop): {flattened_loop}")
print(f"Flattened (comprehension): {flattened_comp}")
```

Chapter 3: Python Data Structures

Python provides several built-in data structures (or collections) for storing and organizing data, each with distinct characteristics and use cases.

3.1 Lists: Ordered, Mutable Sequences

Lists are one of Python's most versatile sequence types. They are defined by enclosing comma-separated items within square brackets ``. Lists are *ordered*, meaning items maintain their position, and *mutable*, meaning their contents can be changed after creation.[9]

Creation:

Python

```
empty_list =
numbers = [1, 2, 3, 4, 5]
mixed_list =] # Lists can store different types

print(f"Empty list: {empty_list}")
print(f"Numbers list: {numbers}")
```

```
print(f"Mixed list: {mixed_list}")
```

Accessing Elements:

- **Indexing:** Access individual items using zero-based indices. Negative indices count from the end (-1 is the last item).[9]
  Python
  ```
  print(f"First number: {numbers}")      # Output: 1
  print(f"Third number: {numbers[2]}")      # Output: 3
  print(f"Last item in mixed: {mixed_list[-1]}") # Output: [1, 2]
  ```

- **Slicing:** Access a sub-list using [start:stop:step]. stop index is exclusive.[9]
  Python
  ```
  print(f"Numbers from index 1 to 3: {numbers[1:4]}") # Output: [2, 3, 4]
  print(f"First three numbers: {numbers[:3]}")       # Output: [1, 2, 3]
  print(f"Numbers from index 2 onwards: {numbers[2:]}") # Output: [3, 4, 5]
  print(f"Every second number: {numbers[::2]}")       # Output: [1, 3, 5]
  print(f"Reversed numbers: {numbers[::-1]}")         # Output: [5, 4, 3, 2, 1]
  ```

Modifying Lists: Due to their mutability, lists can be altered in place.[9]

Python

```
numbers = [1, 2, 3, 4, 5]
print(f"Original numbers: {numbers}")

# Change an element
numbers[2] = 30
print(f"After changing index 2: {numbers}") # Output: [1, 2, 30, 4, 5]

# Change a slice
numbers[1:3] =
print(f"After changing slice [1:3]: {numbers}") # Output:

# Remove elements using slicing
numbers[1:3] =
print(f"After removing slice [1:3]: {numbers}") # Output: [1, 4, 5]
```

```
# Remove elements using del
del numbers[1]
print(f"After deleting index 1: {numbers}") # Output: [1, 5]
```

Common List Methods: Lists have numerous built-in methods for manipulation.[9]

Method	Description	Mutates List?	Returns	Example
append(x)	Adds item x to the end of the list.	Yes	None	my_list.append(6)
extend(iterable)	Appends all items from iterable to the end.	Yes	None	my_list.extend([7])
insert(i, x)	Inserts item x at index i.	Yes	None	my_list.insert(1, 10)
remove(x)	Removes the *first* occurrence of item x. Raises ValueError if not found.	Yes	None	my_list.remove(3)
pop([i])	Removes and returns item at index i (default: last). Raises IndexError.	Yes	Removed item	last = my_list.pop() first = my_list.pop(0)
clear()	Removes all items from the list.	Yes	None	my_list.clear()
index(x[, start[, end]])	Returns the index of the *first* occurrence of x. Raises	No	Index (int)	idx = my_list.index(4)

	ValueError.			
count(x)	Returns the number of times x appears in the list.	No	Count (int)	c = my_list.count(2)
sort(key=None, reverse=False)	Sorts the list in place.	Yes	None	my_list.sort() my_list.sort(reverse=True)
reverse()	Reverses the order of elements in the list in place.	Yes	None	my_list.reverse()
copy()	Returns a *shallow* copy of the list. Equivalent to [:].	No	New list object	new_list = my_list.copy()

Shallow Copies: The mutability of lists requires careful consideration when copying. Assigning a list to a new variable (new_list = old_list) creates a reference, not a copy; both variables point to the same list object. The copy() method or slicing [:] creates a *shallow* copy – a new list object is created, but if the list contains other mutable objects (like nested lists), those nested objects are still shared references.[8] For completely independent copies, including nested structures, the copy.deepcopy() function is needed.

Python

```
list1 = [3, 4]]
list2 = list1.copy() # Shallow copy

list2 = 100
list2.[2]append(5) # Modifying nested list affects both

print(f"List 1: {list1}") # Output: List 1: [3, 4, 5]]
```

```
print(f"List 2: {list2}") # Output: List 2: [3, 4, 5]]

import copy
list3 = copy.deepcopy(list1) # Deep copy
list3.[2]append(6)

print(f"List 1 (after deep copy mod): {list1}") # Output: List 1 (after deep copy mod): [3, 4, 5]]
print(f"List 3 (deep copy): {list3}")        # Output: List 3 (deep copy): [3, 4, 5, 6]]
```

Iteration: Lists are commonly iterated using for loops.[9]

Python

```
numbers = [11, 12, 10]
for num in numbers:
    print(num * 2)

for i, num in enumerate(numbers):
    print(f"Index {i}: {num}")
```

List Comprehensions: A concise syntax for creating lists.[9]

Python

```
# Squares of numbers 0-9
squares = [x**2 for x in range(10)]
print(f"Squares: {squares}") # Output:

# Even numbers from 0-9
evens = [x for x in range(10) if x % 2 == 0]
print(f"Evens: {evens}") # Output:

# Nested list comprehension (transpose a matrix)
matrix = [[1, 2], [3, 4], [5, 6]]
transposed = [[row[i] for row in matrix] for i in range(2)]
```

```
print(f"Transposed matrix: {transposed}") # Output: [[1, 3, 5], [2, 4, 6]]
```

Lists as Stacks (LIFO): Use append() to push and pop() (with no index) to pop.[9]

Python

```
stack =
stack.append('A')
stack.append('B')
stack.append('C')
print(f"Stack: {stack}") # Output:
item = stack.pop()
print(f"Popped: {item}, Stack now: {stack}") # Output: Popped: C, Stack now:
```

Lists as Queues (FIFO): While possible using insert(0, item) or pop(0), these operations are inefficient for lists as they require shifting elements. For efficient queues, collections.deque is strongly recommended.[9]

Python

```
from collections import deque

queue = deque(["Eric", "John", "Michael"])
queue.append("Terry")    # Add to the right
queue.append("Graham")   # Add to the right
print(f"Queue: {queue}") # Output: deque()

first_out = queue.popleft() # Remove from the left (efficient)
print(f"Served: {first_out}, Queue now: {queue}") # Output: Served: Eric, Queue now: deque()
```

3.2 Tuples: Ordered, Immutable Sequences

Tuples are similar to lists in that they are ordered sequences of items. However, their defining characteristic is *immutability* – once a tuple is created, its contents cannot be

changed.[8] Tuples are defined using parentheses () with comma-separated items.

Creation:

```python
empty_tuple = ()
single_item_tuple = (42,) # Note the trailing comma!
coordinates = (10.5, -3.2)
mixed_tuple = (1, "config", True)

print(f"Empty tuple: {empty_tuple}")
print(f"Single item tuple: {single_item_tuple}")
print(f"Coordinates: {coordinates}")
print(f"Mixed tuple: {mixed_tuple}")

# Parentheses are sometimes optional (tuple packing)
packed_tuple = 10, 20, 'thirty'
print(f"Packed tuple: {packed_tuple}, Type: {type(packed_tuple)}")
```

Creating a single-element tuple requires a trailing comma to distinguish it from a value enclosed in parentheses for grouping.[10]

Accessing Elements: Access works identically to lists using indexing and slicing.[10]

```python
print(f"First coordinate: {coordinates}")  # Output: 10.5
print(f"Last item in mixed: {mixed_tuple[-1]}") # Output: True
print(f"Slice of mixed: {mixed_tuple[1:]}")  # Output: ('config', True)
```

Immutability: Attempting to modify a tuple element raises a TypeError.[10]

Python

```
# coordinates = 5.0 # This would raise TypeError: 'tuple' object does not support item assignment
```

While tuples themselves are immutable, if a tuple contains a mutable object (like a list), that mutable object can still be changed in place.

Python

```
mutable_in_tuple = (1, 2, [3, 4])
mutable_in_tuple.[2]append(5)
print(f"Tuple with modified list: {mutable_in_tuple}") # Output: (1, 2, [3, 4, 5])
```

New tuples can be created by concatenating existing tuples.[10]

Python

```
tup1 = (1, 2)
tup2 = (3, 4)
tup3 = tup1 + tup2
print(f"Concatenated tuple: {tup3}") # Output: (1, 2, 3, 4)
```

Common Tuple Methods/Operations: Tuples have fewer methods than lists due to immutability.[10]

Operation/Function	Description	Example	Output
t1 + t2	Concatenation: Creates a new tuple containing elements of t1 then t2.	(1, 2) + (3, 4)	(1, 2, 3, 4)

t * n	Repetition: Creates a new tuple repeating elements of t, n times.	('Hi',) * 3	('Hi', 'Hi', 'Hi')
len(t)	Returns the number of items in the tuple.	len((10, 20, 30))	3
t.count(x)	Returns the number of times item x appears in the tuple.	(1, 2, 2, 3).count(2)	2
t.index(x)	Returns the index of the first occurrence of item x. Raises ValueError.	('a', 'b', 'c').index('b')	1
x in t	Membership test: Returns True if x is in t, False otherwise.	3 in (1, 2, 3)	True
max(t), min(t)	Returns the maximum or minimum item in the tuple (items must be comparable).	max((1, 5, 2))	5
sorted(t)	Returns a *new sorted list* from the tuple's elements.	sorted((3, 1, 2))	[1]

Iteration: Tuples can be iterated using for loops, similar to lists.[10]

Python

```
for item in mixed_tuple:
    print(f"Item: {item}")
```

Packing and Unpacking: Assigning comma-separated values creates a tuple (packing). Assigning a tuple to comma-separated variables unpacks the values.[10]

Python

```
# Packing
point = 10, 20
print(f"Packed point: {point}") # Output: (10, 20)

# Unpacking
x, y = point
print(f"Unpacked: x={x}, y={y}") # Output: x=10, y=20

# Common use: Swapping variables
a, b = 5, 10
a, b = b, a # Packing (10, 5) then unpacking
print(f"Swapped: a={a}, b={b}") # Output: a=10, b=5
```

Tuple unpacking is frequently used for multiple assignments and is the mechanism behind functions returning multiple values.[10]

Use Cases:

- **Data Integrity:** Representing fixed collections where modification is undesirable (e.g., RGB colors, coordinates).
- **Dictionary Keys:** Immutable tuples can be used as keys in dictionaries, unlike mutable lists.[10]
- **Returning Multiple Values:** Functions naturally return multiple values packed as a tuple.[10]
- **String Formatting:** Older string formatting methods used tuples.

3.3 Dictionaries: Unordered (Pre-3.7) / Ordered (3.7+), Mutable Key-Value Pairs

Dictionaries are Python's implementation of associative arrays or hash maps. They store data as *key-value pairs*. Keys must be unique within a dictionary and must be of an immutable (hashable) type (e.g., strings, numbers, tuples containing only immutable elements).[33] Values can be of any type and can be duplicated. Dictionaries are mutable. As of Python 3.7, standard dictionaries remember the order of item insertion.[34]

Creation:

Python

```
empty_dict = {}
student_grades = {"Alice": 85, "Bob": 92, "Charlie": 78}
config = dict(port=8080, host="localhost", debug=True) # Using dict() constructor
inventory = dict.fromkeys(["apple", "banana", "orange"], 0) # Using fromkeys

print(f"Empty dict: {empty_dict}")
print(f"Student grades: {student_grades}")
print(f"Config: {config}")
print(f"Inventory: {inventory}")
```

Accessing Values:

- **By Key ``:** Retrieves the value associated with the key. Raises KeyError if the key doesn't exist.[4]
  Python
  ```
  print(f"Bob's grade: {student_grades}") # Output: 92
  # print(student_grades['David']) # Would raise KeyError
  ```

- **Using .get(key[, default]):** Retrieves the value. Returns None (or the specified default) if the key is not found, avoiding KeyError.[4]
  Python
  ```
  print(f"Alice's grade: {student_grades.get('Alice')}") # Output: 85
  print(f"David's grade: {student_grades.get('David')}") # Output: None
  print(f"Eve's grade (default 0): {student_grades.get('Eve', 0)}") # Output: 0
  ```

Adding/Updating Pairs:

Python

```
student_grades = {"Alice": 85, "Bob": 92}
print(f"Original grades: {student_grades}")
```

```
# Add a new entry
student_grades["Charlie"] = 78
print(f"After adding Charlie: {student_grades}")

# Update an existing entry
student_grades["Alice"] = 88
print(f"After updating Alice: {student_grades}")

# Using update() to merge dictionaries or iterables
new_data = {"Bob": 95, "David": 80}
student_grades.update(new_data)
print(f"After update(): {student_grades}") # Bob updated, David added

student_grades.update([("Eve", 90)]) # Update with iterable
print(f"After update() with iterable: {student_grades}")
```

Why Dictionaries are Fast: Dictionaries offer fast lookups (average O(1) time) because they are implemented using hash tables.[33] When accessing or inserting an item, Python calculates a hash value from the key, which directly (or indirectly after collision resolution) determines the memory location for the associated value. This avoids iterating through all elements as required when searching a list.[4]

Immutable Keys: The hash table implementation necessitates that keys be immutable.[4] If a key were mutable, its hash value could change after insertion, making it impossible to reliably locate the associated value later. This is why lists or other dictionaries cannot be used as keys, while strings, numbers, and tuples (containing only immutables) are valid keys.

Common Dictionary Methods:

Method	Description	Mutates Dict?	Returns	Example
get(key[, default])	Returns the value for key. If key not found, returns default (or None).	No	Value or default	val = d.get('a', 0)
keys()	Returns a dynamic *view* object	No	View object (dict_keys)	k_view = d.keys()

	containing the dictionary's keys.			
values()	Returns a dynamic *view* object containing the dictionary's values.	No	View object (dict_values)	v_view = d.values()
items()	Returns a dynamic *view* object containing the dictionary's (key, value) tuple pairs.	No	View object (dict_items)	i_view = d.items()
pop(key[, default])	Removes key and returns its value. Raises KeyError if key not found and default not given.	Yes	Removed value or default	val = d.pop('b')
popitem()	Removes and returns an arbitrary (key, value) pair (LIFO in Python 3.7+). Raises KeyError if empty.	Yes	(key, value) tuple	k, v = d.popitem()
update(other)	Updates the dictionary with key/value pairs from other (dict or iterable), overwriting existing keys.	Yes	None	d.update({'c': 3}) d.update(d2)
clear()	Removes all	Yes	None	d.clear()

	items from the dictionary.			
setdefault(key[, default])	Returns d[key] if key exists. If not, inserts key with value default (or None) and returns it.	Yes	Value	val = d.setdefault('a', 1)
copy()	Returns a *shallow* copy of the dictionary.	No	New dictionary object	new_d = d.copy()
fromkeys(seq[, value])	Class method: Creates a new dictionary with keys from seq and values set to value (default None).	N/A	New dictionary object	new_d = dict.fromkeys(['x','y'], 0)

Dictionary Views: Methods like .keys(), .values(), and .items() return *view objects*, not lists.[33] Views are dynamic; they reflect subsequent changes made to the dictionary. Iterating over a view while modifying the dictionary requires caution. To get a static list, explicitly convert the view (e.g., list(my_dict.keys())).[35]

Iteration:

```python
student_grades = {"Alice": 88, "Bob": 95, "Charlie": 78}

# Iterate over keys (default)
print("\nIterating over keys:")
for student in student_grades:
    print(f"- {student} has grade {student_grades[student]}")

# Iterate over values
```

```
print("\nIterating over values:")
for grade in student_grades.values():
    print(f"- Grade: {grade}")

# Iterate over items (key-value pairs) - Recommended
print("\nIterating over items:")
for student, grade in student_grades.items():
    print(f"- {student}: {grade}")
```

Dictionary Comprehensions: Concise syntax for creating dictionaries.[34]

Python

```
# Squares dictionary
squares_dict = {x: x*x for x in range(1, 6)}
print(f"\nSquares dict: {squares_dict}") # Output: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Filter dictionary based on value
high_grades = {student: grade for student, grade in student_grades.items() if grade >= 90}
print(f"High grades: {high_grades}") # Output: {'Bob': 95}
```

Nested Dictionaries: Dictionaries can contain other dictionaries as values.[34]

Python

```
users = {
    "user1": {"name": "Alice", "email": "alice@example.com"},
    "user2": {"name": "Bob", "email": "bob@example.com", "admin": True}
}

print(f"\nUser1's email: {users['user1']['email']}")
print(f"Is User2 admin? {users['user2'].get('admin', False)}")

print("\nAll users:")
```

```
for user_id, user_info in users.items():
    print(f" ID: {user_id}")
    for key, value in user_info.items():
        print(f"  {key}: {value}")
```

3.4 Sets: Unordered, Mutable Collections of Unique Elements

Sets are unordered collections of unique, immutable elements. They are mutable themselves (elements can be added or removed), but their elements must be hashable (like numbers, strings, or tuples).[36] Sets are highly optimized for membership testing and removing duplicate elements, leveraging hash tables internally, similar to dictionaries.[37]

Creation:

Python

```
empty_set = set() # Must use set() for an empty set
numbers_set = {1, 2, 3, 4, 4, 5} # Duplicates (4) are ignored
chars_set = set("hello world") # Creates set from unique characters

print(f"Empty set: {empty_set}")
print(f"Numbers set: {numbers_set}") # Output: {1, 2, 3, 4, 5} (order may vary)
print(f"Chars set: {chars_set}")     # Output: {'o', 'w', 'r', 'd', 'l', ' ', 'h', 'e'} (order may vary)

# {} creates an empty dictionary, not a set
# empty_set_wrong = {}
# print(type(empty_set_wrong)) # <class 'dict'>
```

Adding/Removing Elements:

Python

```
my_set = {1, 2, 3}
print(f"Original set: {my_set}")
```

```
# Add an element
my_set.add(4)
my_set.add(2) # Adding existing element does nothing
print(f"After add(4) and add(2): {my_set}") # Output: {1, 2, 3, 4}

# Remove an element (raises KeyError if not found)
my_set.remove(3)
print(f"After remove(3): {my_set}") # Output: {1, 2, 4}
# my_set.remove(10) # Would raise KeyError

# Discard an element (does nothing if not found)
my_set.discard(1)
my_set.discard(10) # No error if element doesn't exist
print(f"After discard(1) and discard(10): {my_set}") # Output: {2, 4}

# Pop an arbitrary element (raises KeyError if empty)
popped_element = my_set.pop()
print(f"Popped element: {popped_element}, Set now: {my_set}")

# Clear all elements
my_set.clear()
print(f"After clear(): {my_set}") # Output: set()
```

The difference between remove() and discard() provides flexibility: use remove() when the element's absence is an error, and discard() when you simply want to ensure the element is not present.[36]

Set Operations: Sets support standard mathematical operations.[36]

Operation	Operator	Method	Description	Example (s1={1,2,3}, s2={3,4,5})	Result
Union	`\`	``	union()	All elements from both sets.	`s1 \`
Intersection	&	intersection()	Elements common to both sets.	s1 & s2	{3}

Difference	-	difference()	Elements in the first set but not the second.	s1 - s2	{1,2}
Symmetric Difference	^	symmetric_difference()	Elements in either set, but not both.	s1 ^ s2	{1,2,4,5}
Subset Check	<=	issubset()	Is the first set a subset of the second?	{1,2} <= s1	True
Proper Subset Check	<	N/A	Is the first set a proper subset of the second?	{1,2} < s1	True
Superset Check	>=	issuperset()	Is the first set a superset of the second?	s1 >= {1,2}	True
Proper Superset Check	>	N/A	Is the first set a proper superset of the second?	s1 > {1,2}	True
Disjoint Check	N/A	isdisjoint()	Do the sets have no elements in common?	s1.isdisjoint({4, 5})	True

Examples of Set Operations:

Python

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}
```

```
set_c = {1, 2}

print(f"Set A: {set_a}")
print(f"Set B: {set_b}")
print(f"Set C: {set_c}")

print(f"Union (A | B): {set_a | set_b}")
print(f"Intersection (A & B): {set_a & set_b}")
print(f"Difference (A - B): {set_a - set_b}")
print(f"Difference (B - A): {set_b - set_a}")
print(f"Symmetric Difference (A ^ B): {set_a ^ set_b}")

print(f"Is C subset of A? (C <= A): {set_c <= set_a}") # True
print(f"Is A superset of C? (A >= C): {set_a >= set_c}") # True
print(f"Is A disjoint from {{7, 8}}? {set_a.isdisjoint({7, 8})}") # True
print(f"Is A disjoint from B? {set_a.isdisjoint(set_b)}") # False
```

Set Comprehensions: Concise syntax for creating sets.[30]

Python

```
# Set of squares
squares_set = {x**2 for x in range(5)}
print(f"Squares set: {squares_set}") # Output: {0, 1, 4, 9, 16}

# Set of unique uppercase letters from a string
text = "Hello World"
uppercase_letters = {char for char in text if char.isupper()}
print(f"Uppercase letters: {uppercase_letters}") # Output: {'W', 'H'}
```

Chapter 4: Error and Exception Handling

Effective programs must anticipate and manage errors that can occur during execution. Python distinguishes between two main categories of errors.

4.1 Introduction to Errors

- **Syntax Errors:** These are errors in the structure or grammar of the Python code

itself (e.g., misspelled keywords, incorrect indentation, missing colons). The Python parser detects these *before* the program begins execution, preventing the script from running at all.

```
Python
# Syntax Error Example (missing colon)
# if x > 5
#     print("x is large")
```

- **Exceptions:** These errors occur *during* program execution (runtime) after the syntax has been verified. They arise when an operation is syntactically correct but logically invalid in the current context (e.g., trying to divide by zero, accessing a non-existent list index or dictionary key, attempting an operation on incompatible data types).[39] If not handled, exceptions cause the program to terminate and print a traceback.

Common Built-in Exceptions:

- TypeError: Operation applied to an object of inappropriate type (e.g., len(123), '2' + 2).[12]
- NameError: Using a variable or function name that hasn't been defined.[10]
- IndexError: Accessing a sequence (list, tuple, string) index that is out of range.[9]
- KeyError: Accessing a dictionary key that doesn't exist.[4]
- ValueError: Function receives an argument of the correct type but an inappropriate value (e.g., int('abc'), math.sqrt(-1)).[9]
- ZeroDivisionError: Attempting division or modulo by zero.
- FileNotFoundError: Trying to open a file that does not exist in read mode.
- AttributeError: Attempting to access an attribute or method that doesn't exist for an object (e.g., .[1]appendd(3)).

4.2 Handling Exceptions: try...except

The try...except block allows you to anticipate and handle specific exceptions gracefully, preventing program crashes and enabling alternative actions.[2]

Syntax:

```
Python

try:
```

```
    # Code block where exceptions might occur
    pass
except SpecificExceptionType1:
    # Code block to handle SpecificExceptionType1
    pass
except (SpecificExceptionType2, SpecificExceptionType3):
    # Code block to handle either SpecificExceptionType2 or SpecificExceptionType3
    pass
except ExceptionType4 as e:
    # Code block to handle ExceptionType4, accessing the exception object as 'e'
    # print(f"Caught an error: {e}")
    pass
except:
    # Catch any other exception (generally discouraged unless logging/re-raising)
    pass
```

Example: Handling potential errors during division.

Python

```
try:
    numerator_str = input("Enter the numerator: ")
    denominator_str = input("Enter the denominator: ")

    numerator = float(numerator_str)
    denominator = float(denominator_str)

    result = numerator / denominator
    print(f"The result is: {result}")

except ValueError:
    print("Error: Please enter valid numbers for numerator and denominator.")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
except Exception as e: # Catch any other unexpected error
    print(f"An unexpected error occurred: {e}")
```

```
    print(f"Type of error: {type(e)}")
```

Handling specific exceptions allows for tailored error messages or recovery strategies. Catching a general Exception can be useful as a fallback but should be used cautiously, as it can mask unexpected problems. It's generally better practice to catch the specific exceptions you anticipate.[39]

4.3 The else Clause in try...except

Similar to loops, try...except blocks can have an optional else clause. The code within the else block executes *only if* the try block completes successfully without raising any exceptions.[6] This is useful for code that should run only when the potentially problematic operations in the try block succeed.

Example: Processing file content only if opening succeeds.

Python

```
file_path = "data.txt"
try:
    print(f"Attempting to open '{file_path}'...")
    file_handle = open(file_path, 'r')
except FileNotFoundError:
    print(f"Error: File '{file_path}' not found.")
except PermissionError:
    print(f"Error: Permission denied to read '{file_path}'.")
else:
    # This block runs only if the 'try' block succeeded
    print("File opened successfully.")
    try:
        content = file_handle.read()
        print("File content read:")
        print(content[:100] + "..." if len(content) > 100 else content) # Print first 100 chars
    except Exception as e:
        print(f"Error reading file content: {e}")
    finally:
        # Ensure file is closed even if reading fails
        file_handle.close()
```

```
        print("File closed in else's finally.")
```

4.4 The finally Clause

The finally block is optional and, if present, *always* executes after the try, except, and else blocks have finished, regardless of whether an exception occurred, was caught, or if the try block completed successfully.[40] Its primary purpose is for cleanup actions that must happen unconditionally, such as closing files, releasing network connections, or freeing other resources, thereby preventing resource leaks.[39]

Example: Ensuring a resource is released.

Python

```
resource_acquired = False
try:
    print("Acquiring resource...")
    # Simulate acquiring a resource (e.g., opening a file, network connection)
    resource_acquired = True
    print("Resource acquired.")

    # Simulate an operation that might fail
    # if random.choice():
    #     raise ValueError("Simulated operation failure")

    print("Operation successful.")

except Exception as e:
    print(f"An error occurred during operation: {e}")

finally:
    # This block always runs
    print("Executing finally block...")
    if resource_acquired:
        # Simulate releasing the resource
        print("Releasing resource.")
    else:
```

```
    print("No resource was acquired.")
```

The pattern of acquiring a resource, performing actions in a try block, and ensuring release in a finally block is so common that Python provides the with statement (discussed in File I/O) as a more concise way to manage resources that support the context management protocol (__enter__ and __exit__ methods).[40]

4.5 Raising Exceptions (raise)

You can manually trigger exceptions using the raise keyword. This is useful for signaling errors based on specific conditions in your code, validating input, or creating custom error types. You can raise built-in exception types or instances of custom exception classes (derived from Exception).

Example: Validating function input.

Python

```
def calculate_discount(price, percentage):
    if not isinstance(price, (int, float)) or not isinstance(percentage, (int, float)):
        raise TypeError("Price and percentage must be numeric.")
    if price < 0:
        raise ValueError("Price cannot be negative.")
    if not (0 <= percentage <= 100):
        raise ValueError("Percentage must be between 0 and 100.")

    discount_amount = price * (percentage / 100)
    return price - discount_amount

try:
    final_price = calculate_discount(100, 110) # Invalid percentage
    # final_price = calculate_discount("abc", 10) # Invalid type
    # final_price = calculate_discount(50, 10) # Valid
    print(f"Final price: ${final_price:.2f}")
except (TypeError, ValueError) as e:
    print(f"Error calculating discount: {e}")
```

Raising appropriate exceptions makes function behavior clearer and allows calling code to handle specific error conditions effectively.

Chapter 5: Functions

Functions are fundamental building blocks in Python, allowing code to be organized into reusable, named blocks. They improve modularity, readability, and maintainability.

5.1 Defining and Calling Functions

Functions are defined using the def keyword, followed by the function name, parentheses () enclosing any parameters, and a colon :. The indented block following the colon is the function body.[6]

Python

```
# Function definition
def greet(name):
    """This function greets the person passed in as a parameter.""" # Docstring
    print(f"Hello, {name}!")

def calculate_area(length, width):
    """Calculates the area of a rectangle."""
    area = length * width
    return area

# Function calls
greet("Alice") # Output: Hello, Alice!

rectangle_area = calculate_area(10, 5)
print(f"The area is: {rectangle_area}") # Output: The area is: 50
```

The string literal immediately following the function header is the *docstring*, used to document the function's purpose.[6]

5.2 Parameters and Arguments

- **Parameters:** Variables listed inside the parentheses in the function definition.[32]
- **Arguments:** Actual values passed to the function when it is called.[32]

Python offers flexible ways to define parameters and pass arguments:

- **Positional Arguments:** Arguments are matched to parameters based on their order.[32]
  Python
  ```
  def describe(item, color):
      print(f"The {item} is {color}.")

  describe("sky", "blue") # 'sky' -> item, 'blue' -> color
  ```

- **Keyword Arguments:** Arguments are passed using parameter_name=value syntax. The order doesn't matter if names are specified.[6]
  Python
  ```
  describe(color="red", item="apple") # Order changed, but mapping is explicit
  ```

- **Default Parameter Values:** Parameters can have default values, used if no argument is provided for them during the call.[6] Default parameters must follow non-default parameters in the definition.
  Python
  ```
  def power(base, exponent=2):
      return base ** exponent

  print(power(5))     # Output: 25 (exponent defaults to 2)
  print(power(5, 3))  # Output: 125 (exponent is provided)
  ```

  Important Note on Mutable Defaults: Default values are evaluated *only once* when the function is defined. If a default value is a mutable object (like a list or dictionary), modifications to it within the function will persist across subsequent calls, often leading to unexpected behavior.[6] The standard idiom is to use None as the default and create the mutable object inside the function if needed.[6]
  Python
  ```
  # Problematic mutable default:
  def add_item_bad(item, my_list=):
      my_list.append(item)
      return my_list

  print(add_item_bad(1)) # Output: [1]
  print(add_item_bad(2)) # Output: [1, 2] - list persists!

  # Correct pattern:
  def add_item_good(item, my_list=None):
  ```

```
    if my_list is None:
        my_list =
    my_list.append(item)
    return my_list

print(add_item_good(1)) # Output: [1]
print(add_item_good(2)) # Output: [2] - new list created each time
print(add_item_good(3, [11, 12])) # Output: [11, 12, 3] - can still pass a list
```

- **Arbitrary Positional Arguments (*args):** Collects any extra positional arguments into a tuple.[6]

```
Python
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_all(1, 2, 3))       # Output: 6
print(sum_all(10, 20, 30, 40)) # Output: 100
```

- **Arbitrary Keyword Arguments (**kwargs):** Collects any extra keyword arguments into a dictionary.[6]

```
Python
def print_info(**info):
    print("Information received:")
    for key, value in info.items():
        print(f"- {key}: {value}")

print_info(name="Alice", age=30, city="New York")
# Output:
# Information received:
# - name: Alice
# - age: 30
# - city: New York
```

*args and **kwargs provide significant flexibility, allowing functions to accept a variable number of inputs, which is particularly useful in scenarios like function decorators or wrappers.[6]

5.3 The return Statement

The return statement exits a function and optionally sends a value (or values) back to the caller code.[32]

- **Returning a Single Value:**
  Python
  ```
  def multiply(a, b):
      return a * b

  product = multiply(6, 7)
  print(f"Product: {product}") # Output: 42
  ```

- **Returning Multiple Values:** Python makes returning multiple values syntactically straightforward by automatically packing them into a tuple. The caller can then unpack this tuple.[32]
  Python
  ```
  import statistics

  def basic_stats(data):
      mean = statistics.mean(data)
      median = statistics.median(data)
      stdev = statistics.stdev(data)
      return mean, median, stdev # Returns a tuple

  sample_data = [1, 5, 2, 8, 7, 3, 4, 6, 9]
  m, med, sd = basic_stats(sample_data) # Unpacking the returned tuple
  print(f"Mean: {m}, Median: {med}, StDev: {sd:.2f}")

  stats_tuple = basic_stats(sample_data) # Receiving as a tuple
  print(f"Stats tuple: {stats_tuple}")
  ```

- **Implicit return None:** If a function reaches its end without encountering a return statement, or if return is used without a value, it implicitly returns None.[32]
  Python
  ```
  def log_message(message):
      print(f"LOG: {message}")
      # No explicit return

  result = log_message("System started.")
  print(f"Return value: {result}") # Output: Return value: None
  ```

5.4 Scope: Local and Global Variables

Scope defines the accessibility of variables within different parts of a program.[2] Python follows the LEGB rule for name resolution: Local, Enclosing function locals, Global, Built-in.

- **Local Scope:** Variables defined inside a function are local to that function and cannot be accessed from outside.[32]
  Python
  ```
  def my_func():
      local_var = "I am local"
      print(local_var)

  my_func()
  # print(local_var) # Would cause NameError
  ```

- **Global Scope:** Variables defined outside any function are global and can be accessed (read) from anywhere, including inside functions.[32]
  Python
  ```
  global_var = "I am global"

  def read_global():
      print(f"Inside function: {global_var}")

  read_global()
  print(f"Outside function: {global_var}")
  ```

- **Modifying Globals (global keyword):** To modify a global variable from within a function, you must explicitly declare it using the global keyword. However, modifying global variables is generally discouraged as it can make code harder to track and debug.[32] Prefer passing values as arguments and returning results.
  Python
  ```
  counter = 0

  def increment_global_counter():
      global counter # Declare intent to modify the global variable
      counter += 1
      print(f"Counter inside function: {counter}")

  increment_global_counter()
  ```

```
print(f"Counter outside function: {counter}")
```

5.5 Lambda Functions

Lambda functions provide a concise way to create small, anonymous functions (functions without a formal name defined using def).[6] They are defined using the lambda keyword.

Syntax:

```
Python

lambda arguments: expression
```

Lambda functions can have any number of arguments but are restricted to a single expression. The value of this expression is implicitly returned.[32]

Examples:

```
Python

# Simple lambda adding 10
add_ten = lambda x: x + 10
print(add_ten(5)) # Output: 15

# Lambda with multiple arguments
multiply = lambda x, y: x * y
print(multiply(4, 7)) # Output: 28

# Using lambda with sorted() as a key function
points = [(1, 5), (3, 2), (2, 8)]
points_sorted_by_y = sorted(points, key=lambda point: point[1])
print(f"Points sorted by y-coordinate: {points_sorted_by_y}")
# Output: [(3, 2), (1, 5), (2, 8)]
```

```
# Using lambda with filter()
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(f"Even numbers: {even_numbers}") # Output: [2, 4, 6]

# Using lambda with map()
numbers = [1, 2, 3, 4]
squared_numbers = list(map(lambda x: x**2, numbers))
print(f"Squared numbers: {squared_numbers}") # Output: [1, 4, 9, 13]
```

Their limitation to a single expression makes lambdas suitable for simple, short-lived operations, often where a function object is required as an argument to another function (like sorted, map, filter), but regular def functions are necessary for more complex logic.[32]

Chapter 6: Modules and Packages

Modules and packages are Python's way of organizing code into larger units, promoting reusability and maintainability.

6.1 Importing Modules

A **module** is simply a Python file (.py) containing Python definitions and statements (functions, classes, variables). Modules allow code to be logically grouped and reused across different scripts.[43]

To use the code defined in a module, you must **import** it using the import statement. Python offers several ways to import:

- **import module_name:** Imports the entire module. Access its contents using dot notation (module_name.function_name).[43] This keeps the module's contents within their own namespace, preventing naming conflicts with the importing script's names. This is generally the recommended approach for clarity.
  Python
  ```
  import math

  print(math.pi)
  print(math.sqrt(16))
  ```

- **from module_name import name1, name2:** Imports specific names (functions, classes, variables) directly into the importing script's namespace. They can then be used without the module prefix.[43] Use cautiously, as it can lead to naming

collisions if the imported name already exists locally.

```python
from math import pi, sqrt

print(pi)
print(sqrt(16))
```

- **import module_name as alias:** Imports the entire module but gives it a shorter or more convenient alias in the local namespace.[43]

```python
import math as m

print(m.pi)
print(m.sqrt(16))
```

- **from module_name import name as alias:** Imports a specific name and gives it an alias locally.[43]

```python
from math import pi as PI_VALUE

print(PI_VALUE)
```

- **from module_name import *:** Imports *all* names (except those starting with _) from the module directly into the local namespace. This is generally **discouraged** because it makes it unclear where names originated and significantly increases the risk of naming conflicts.[44]

```python
# Generally avoid this:
# from math import *
# print(pi)
# print(sqrt(16))
```

6.2 Standard Library Examples: math and random

Python comes with a rich **standard library** – a collection of modules included with the Python installation. math and random are two commonly used examples.

The math Module: Provides access to mathematical functions and constants, primarily for floating-point numbers, based on the standard C library functions.[48] For complex number mathematics, the cmath module is available.[50]

Function/Constant	Description	Example	Result
math.pi	Mathematical constant π (approx. 3.14159)	math.pi	3.141592653589793
math.e	Mathematical constant e (approx. 2.71828)	math.e	2.718281828459045
math.sqrt(x)	Square root of x (x must be non-negative)	math.sqrt(25)	5.0
math.pow(x, y)	x raised to the power y (returns float)	math.pow(2, 3)	8.0
math.log(x[, base])	Logarithm of x (natural log if base omitted, else to base)	math.log(math.e) math.log(100, 10)	1.0 2.0
math.sin(x)	Sine of x (x in radians)	math.sin(math.pi/2)	1.0
math.cos(x)	Cosine of x (x in radians)	math.cos(math.pi)	-1.0
math.tan(x)	Tangent of x (x in radians)	math.tan(0)	0.0
math.radians(d)	Convert angle d from degrees to radians	math.radians(180)	math.pi
math.degrees(r)	Convert angle r from radians to degrees	math.degrees(math.pi)	180.0
math.ceil(x)	Smallest integer greater than or equal to x	math.ceil(4.2)	5
math.floor(x)	Largest integer less	math.floor(4.8)	4

	than or equal to x		
math.factorial(n)	n! (n must be non-negative integer)	math.factorial(5)	120

Python

```
import math

angle_degrees = 60
angle_radians = math.radians(angle_degrees)

print(f"{angle_degrees} degrees is {angle_radians:.4f} radians")
print(f"Sine of {angle_degrees} degrees: {math.sin(angle_radians):.4f}")
print(f"Ceiling of 9.1: {math.ceil(9.1)}")
print(f"Floor of 9.9: {math.floor(9.9)}")
```

The random Module: Implements pseudo-random number generators for various distributions.[51] "Pseudo-random" means the numbers appear random but are generated by a deterministic algorithm (Mersenne Twister by default) based on an initial state or "seed". This is suitable for simulations, games, and sampling, but for security-sensitive applications requiring true unpredictability, os.urandom() or the secrets module should be used.[51]

Function	Description	Example	Possible Result(s)
random.random()	Random float x, 0.0≤x<1.0	random.random()	0.76...
random.uniform(a, b)	Random float x, a≤x≤b (or b≤x≤a)	random.uniform(1, 10)	5.23...
random.randint(a, b)	Random integer x, a≤x≤b (inclusive)	random.randint(1, 6)	1, 2, 3, 4, 5, 6

random.randrange(start, stop[, step])	Random integer from range(start, stop, step)	random.randrange(0, 10, 2)	0, 2, 4, 6, 8
random.choice(seq)	Random element from a non-empty sequence seq.	random.choice(['a', 'b', 'c'])	'a', 'b', or 'c'
random.shuffle(seq)	Shuffles the sequence seq in place.	lst=[1]; random.shuffle(lst)	lst becomes [3] etc.
random.sample(population, k)	Returns a list of k unique elements chosen from population.	random.sample(range(100), 5)	``
random.seed(a)	Initializes the random number generator for reproducibility.	random.seed(42)	N/A

Python

```
import random

# Set seed for reproducible results (optional)
random.seed(123)

print(f"Random float (0-1): {random.random():.4f}")
print(f"Random integer (1-10): {random.randint(1, 10)}")

options = ["rock", "paper", "scissors"]
print(f"Random choice: {random.choice(options)}")

deck = list(range(1, 5)) # [1, 2, 3, 4]
random.shuffle(deck)
print(f"Shuffled deck: {deck}")

unique_winners = random.sample(range(1, 101), 3) # Pick 3 unique numbers from 1-100
```

```
print(f"Sample winners: {unique_winners}")
```

6.3 Creating and Importing Custom Modules

Any Python file can act as a module. To create a custom module, simply save your Python code (functions, classes, variables) in a file with a .py extension.[44]

Example: Create my_utils.py:

Python

```
# my_utils.py
PI = 3.14159

def circle_area(radius):
    """Calculates the area of a circle."""
    if radius < 0:
        raise ValueError("Radius cannot be negative.")
    return PI * (radius ** 2)

def rectangle_perimeter(length, width):
    """Calculates the perimeter of a rectangle."""
    return 2 * (length + width)

print("my_utils module loaded") # This runs on import

if __name__ == "__main__":
    # This block runs ONLY when the script is executed directly
    # Often used for tests or simple command-line interface
    print("\nRunning my_utils.py as main script:")
    print(f"Area of circle with radius 5: {circle_area(5)}")
    print(f"Perimeter of 5x10 rectangle: {rectangle_perimeter(5, 10)}")
```

To use this module from another script (main_script.py) in the **same directory**:

Python

```
# main_script.py
import my_utils

radius = 7
area = my_utils.circle_area(radius)
print(f"The area of a circle with radius {radius} is {area:.2f}")

# Accessing variable from module
print(f"Value of PI from module: {my_utils.PI}")

perimeter = my_utils.rectangle_perimeter(length=4, width=8)
print(f"Rectangle perimeter: {perimeter}")
```

When main_script.py is run, it first imports my_utils.py. The print("my_utils module loaded") line in my_utils.py executes during the import. The code inside the if __name__ == "__main__": block in my_utils.py does *not* run during import, only if my_utils.py were executed directly.[44] This if __name__ == "__main__": construct is a standard idiom for creating files that can be both imported as modules and run as standalone scripts.[44]

Module Search Path: If the module is not in the same directory, Python searches for it in the directories listed in sys.path. This list includes [46]:

1. The directory containing the input script (or the current directory).
2. Directories listed in the PYTHONPATH environment variable.
3. Installation-dependent default paths.

You can programmatically add a directory to the search path, although managing PYTHONPATH or using virtual environments and packaging (e.g., via pip install -e. for editable installs) are generally preferred for larger projects.[45]

Python

```
# Example: Importing from a different directory (assuming my_utils.py is in../libs)
# import sys
# sys.path.append('../libs') # Add the directory to the path
# import my_utils
```

```
# print(my_utils.circle_area(2))
```

6.4 Introduction to Packages

As projects grow, organizing modules into a single directory becomes unwieldy. **Packages** allow modules to be structured hierarchically using directories.[46] A directory is treated as a package if it contains a file named __init__.py. This file can be empty, but it must exist for Python to recognize the directory as a package.[46]

Example Structure:

```
my_project/
├── main_app.py
└── my_package/
    ├── __init__.py
    ├── module1.py
    └── sub_package/
        ├── __init__.py
        └── module2.py
```

Importing from Packages: Use dot notation corresponding to the directory structure.

Python

```
# In main_app.py

# Import entire module
import my_package.module1
my_package.module1.some_function()

# Import specific function
from my_package.module1 import some_function
some_function()
```

```
# Import submodule
from my_package.sub_package import module2
module2.another_function()

# Import function from submodule
from my_package.sub_package.module2 import another_function
another_function()
```

The __init__.py file can also contain Python code to initialize the package (e.g., set package-level variables) or define what names are imported when from my_package import * is used (by defining an __all__ list), although import * from packages has the same drawbacks as from modules.[46] Packages provide essential structure for large applications and libraries.[46]

Chapter 7: File Input/Output

Interacting with files is a common requirement for storing data, reading configurations, and processing information. Python provides built-in functions and methods for file handling.

7.1 Opening and Closing Files

Before reading from or writing to a file, it must be opened using the built-in open() function. This function returns a file object (also called a file handle).[4] After operations are complete, the file must be closed using the .close() method to ensure data is flushed from buffers to the disk and system resources are released.[4]

open() Syntax:

Python

```
file_object = open(file_path, mode='r', encoding=None)
```

- file_path: String representing the path to the file.
- mode: A string indicating how the file will be used. Common modes include [4]:
  - 'r': Read (default). File must exist.
  - 'w': Write. Creates file if it doesn't exist; truncates (empties) if it does.
  - 'a': Append. Creates file if it doesn't exist; writes data to the end.
  - 'rb': Read binary.

    - 'wb': Write binary.
    - 'ab': Append binary.
    - '+' (appended to mode, e.g., 'r+', 'w+'): Opens for updating (reading and writing).
- encoding: Specifies the encoding for text files (e.g., 'utf-8', 'ascii'). Crucial for correctly interpreting characters, especially outside the basic ASCII range. Often system-dependent if omitted.[4]

Manual Closing (Less Recommended): Using try...finally ensures closure even if errors occur.[55]

Python

```
reader = None # Initialize outside try
try:
    reader = open('config.txt', 'r', encoding='utf-8')
    # Process file...
    print("File opened manually.")
except FileNotFoundError:
    print("Config file not found.")
finally:
    if reader:
        reader.close()
        print("File closed manually in finally block.")
```

Automatic Closing with with (Recommended): The with statement provides context management for file objects. It automatically calls .close() when the block is exited, even if exceptions occur.[4] This is the preferred, more Pythonic, and safer approach.

Python

```
try:
    with open('config.txt', 'r', encoding='utf-8') as config_file:
        print("File opened with 'with'.")
```

```
        # Process file content within this block
        content = config_file.read(50) # Read first 50 chars
        print(f"First 50 chars: {content}")
    # File is automatically closed here, outside the 'with' block
    print("File automatically closed.")
except FileNotFoundError:
    print("Config file not found.")
except Exception as e:
    print(f"An error occurred: {e}")
```

The with statement guarantees cleanup, making code cleaner and more robust compared to manual try...finally blocks for file handling.[40]

7.2 Reading Files

Once a file is opened in a read mode ('r' or 'rb'), several methods can be used to read its content.[55]

- **read(size=-1):** Reads up to size bytes/characters. If size is omitted or negative, reads the entire file content into a single string (text mode) or bytes object (binary mode). Be cautious with large files, as this loads everything into memory.[4]

  Python
  ```
  try:
      with open('example.txt', 'r') as f:
          full_content = f.read()
          print("--- Full Content (read) ---")
          print(full_content)
  except FileNotFoundError:
      print("example.txt not found.")
  ```

- **readline(size=-1):** Reads a single line from the file, including the trailing newline character (\n), up to size bytes/characters if specified. Returns an empty string/bytes object at the end of the file.[55]

  Python
  ```
  try:
      with open('example.txt', 'r') as f:
          print("\n--- First Line (readline) ---")
          line1 = f.readline()
          print(line1, end='') # end='' prevents extra newline
          print("\n--- Second Line (readline) ---")
          line2 = f.readline()
  ```

```
        print(line2, end='')
except FileNotFoundError:
    print("example.txt not found.")
```

- **readlines():** Reads all remaining lines from the file and returns them as a list of strings (text mode) or bytes objects (binary mode), each including the trailing newline. Can consume significant memory for large files.[55]

```
Python
try:
    with open('example.txt', 'r') as f:
        all_lines = f.readlines()
        print("\n--- All Lines (readlines) ---")
        print(all_lines) # Prints list representation
        print("--- Processing lines from list ---")
        for line in all_lines:
            print(line.strip()) # strip() removes leading/trailing whitespace
except FileNotFoundError:
    print("example.txt not found.")
```

- **Iterating over the File Object:** This is the most memory-efficient way to process a file line by line, especially large files. The for loop reads one line at a time as needed.[4]

```
Python
try:
    print("\n--- Iterating Line by Line ---")
    with open('example.txt', 'r') as f:
        for line_number, line in enumerate(f, 1):
            print(f"{line_number}: {line.strip()}")
except FileNotFoundError:
    print("example.txt not found.")
```

  This approach avoids loading the entire file into memory, making it suitable for processing files of any size.[4]

7.3 Writing Files

Files opened in write ('w') or append ('a') mode can be written to using the write() method.[4]

- **write(string):** Writes the given string (text mode) or bytes object (binary mode) to the file. It does *not* automatically add newline characters; these must be included explicitly (\n) if desired.[55] Returns the number of characters/bytes

written.

```
Python
lines_to_write =

# Using 'w' - overwrites file if it exists
try:
    with open('output.txt', 'w', encoding='utf-8') as writer:
        writer.write("This is the beginning.\n")
        for line in lines_to_write:
            writer.write(line)
        writer.write("This is the end.") # No newline at the very end
    print("\nFile 'output.txt' written (or overwritten).")
except Exception as e:
    print(f"Error writing file: {e}")

# Using 'a' - appends to file if it exists
try:
    with open('output.txt', 'a', encoding='utf-8') as appender:
        appender.write("\nAppending this line.")
    print("Appended to 'output.txt'.")
except Exception as e:
    print(f"Error appending to file: {e}")
```

- **Buffered Writing:** File writing is often buffered by the operating system or Python's standard library for efficiency.[55] Data written using write() may not be physically saved to the disk immediately. Buffers are typically flushed when they are full, when file.flush() is called explicitly, or, most importantly, when the file is closed. Using the with statement ensures the file is properly closed and buffers are flushed.[55]

7.4 Working with Binary Files

Binary files (images, audio, executables, etc.) should be handled using binary modes ('rb', 'wb', 'ab') to prevent data corruption caused by encoding/decoding or newline translation issues inherent in text mode.[55] Binary operations work directly with bytes objects.

Example: Reading and writing binary data.

Python

```
# Create a simple binary file
try:
    with open('binary_data.bin', 'wb') as bin_writer:
        # Create a bytes object (sequence of bytes 0-255)
        data_to_write = bytes() # Represents "Hello" in ASCII
        bin_writer.write(data_to_write)
        more_data = b' World' # Another way to create bytes literal
        bin_writer.write(more_data)
    print("\nBinary file 'binary_data.bin' written.")
except Exception as e:
    print(f"Error writing binary file: {e}")

# Read the binary file
try:
    with open('binary_data.bin', 'rb') as bin_reader:
        binary_content = bin_reader.read()
        print(f"Binary content read: {binary_content}")
        # Decode bytes to string if appropriate encoding is known (e.g., ASCII/UTF-8)
        try:
            text_content = binary_content.decode('utf-8')
            print(f"Decoded text content: {text_content}")
        except UnicodeDecodeError:
            print("Could not decode binary content as UTF-8 text.")
except FileNotFoundError:
    print("binary_data.bin not found.")
except Exception as e:
    print(f"Error reading binary file: {e}")
```

Working in binary mode ensures that the exact byte sequence is preserved without interpretation, which is essential for non-text data.[55]

Chapter 8: Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data in the form of fields (often known as attributes or properties) and code in the form of procedures (often known as methods). OOP allows programmers to structure code in a way that models real-world

entities or abstract concepts, promoting organization, reusability, and scalability.[39]

8.1 Classes and Objects

- **Class:** A blueprint or template for creating objects. It defines a set of attributes (data) and methods (functions) that the objects created from the class will have.[39]
- **Object (Instance):** A specific realization created from a class. Each object has its own state (values of its attributes) but shares the structure and behavior (methods) defined by its class.[39] Creating an object from a class is called **instantiation**.

OOP helps bundle related data and the functions that operate on that data together, leading to more modular and understandable code compared to purely procedural approaches where data and functions are often separate.[39]

8.2 Defining a Class (class)

Classes are defined using the class keyword, followed by the class name (typically using CapitalizedWords convention) and a colon. The indented block contains the class definition.[39]

Python

```
class Dog:
    # Class body (attributes and methods go here)
    pass # Use pass for an empty class definition
```

8.3 The Constructor (__init__)

The __init__ method is a special method (a "dunder" or double-underscore method) that acts as the class constructor. It is automatically called when a new object (instance) of the class is created.[39] Its primary role is to initialize the object's attributes.

The first parameter of __init__ (and all instance methods) is conventionally named self. self refers to the specific instance of the class being created or acted upon.[39]

Python

```
class Dog:
    # The constructor
    def __init__(self, name_param, age_param):
        print(f"Creating a new Dog object named {name_param}...")
        # Initialize instance attributes using self
        self.name = name_param
        self.age = age_param
        self.tricks = # Each dog starts with an empty list of tricks

# Instantiating the class calls __init__
dog1 = Dog("Buddy", 5)
dog2 = Dog("Lucy", 3)
```

8.4 Instance Attributes

Instance attributes are variables that belong to a specific object (instance) of a class. Their values represent the state of that particular object and can differ between instances.[39] They are typically defined within the __init__ method using the self.attribute_name = value syntax and accessed using dot notation (object_name.attribute_name).

Python

```
print(f"{dog1.name} is {dog1.age} years old.") # Accessing attributes of dog1
print(f"{dog2.name} is {dog2.age} years old.") # Accessing attributes of dog2

# Attributes can be modified (if not intended to be private)
dog1.age = 6
print(f"{dog1.name} is now {dog1.age} years old.")
```

8.5 Instance Methods

Instance methods are functions defined inside a class that perform actions related to the object's state or capabilities. They always take self as their first parameter, allowing them to access and modify the instance's attributes and call other methods on the same instance.[39]

Python

```
class Dog:
    species = "Canis familiaris" # Class attribute (shared by all instances)

    def __init__(self, name_param, age_param):
        self.name = name_param
        self.age = age_param
        self.tricks =

    # Instance method
    def description(self):
        return f"{self.name} is a {self.species}, aged {self.age}"

    # Another instance method modifying state
    def add_trick(self, trick):
        self.tricks.append(trick)
        print(f"{self.name} learned a new trick: {trick}")

    # Method using other attributes
    def show_tricks(self):
        if not self.tricks:
            print(f"{self.name} knows no tricks yet.")
        else:
            print(f"{self.name}'s tricks: {', '.join(self.tricks)}")

    # Special dunder method for string representation
    def __str__(self):
        return f"Dog(name='{self.name}', age={self.age})"

# Create instances
buddy = Dog("Buddy", 5)
lucy = Dog("Lucy", 3)

# Call instance methods
print(buddy.description())
lucy.add_trick("sit")
```

```
lucy.add_trick("roll over")
buddy.show_tricks()
lucy.show_tricks()

# __str__ is called when printing the object
print(buddy)
print(lucy)
```

The self parameter is the crucial link that allows methods defined in the class blueprint to operate on the specific data contained within each individual object instance.[39] Special methods like __init__ and __str__ (dunder methods) provide hooks into Python's built-in operations (object creation, printing).

Conclusion

This document has provided a comprehensive overview of fundamental Python concepts, progressing from basic syntax and data types to control flow structures, essential data structures (lists, tuples, dictionaries, sets), error handling, functions, modules, file I/O, and an introduction to object-oriented programming.

Python's clear syntax, dynamic typing, extensive standard library (including modules like math and random), and powerful data structures make it a versatile language suitable for a wide array of applications, from simple scripting and automation to complex web development, data science, and scientific computing.[1] Key features like list comprehensions, the with statement for resource management, and flexible function argument handling contribute to its expressiveness and efficiency. The inclusion of robust exception handling mechanisms and the principles of OOP further enable the development of reliable and scalable software.

For continued learning, exploration of specific libraries relevant to particular domains (e.g., NumPy/Pandas for data analysis, Flask/Django for web development, Tkinter/PyQt for GUI development) is recommended. Delving deeper into advanced OOP concepts (inheritance, polymorphism), decorators, generators, and asynchronous programming will further enhance proficiency in Python.[2] Resources like the official Python documentation, tutorials from platforms like Real Python, and community forums provide valuable avenues for ongoing study and problem-solving.[2]

Works cited

1. What is Python? Executive Summary | Python.org, accessed April 30, 2025, https://www.python.org/doc/essays/blurb/

2. Python Beginners — Python Beginners documentation, accessed April 30, 2025, https://python-adv-web-apps.readthedocs.io/
3. How can I learn the basics of Python? – Real Python, accessed April 30, 2025, https://realpython.com/python-basics/
4. Python Dict and File | Python Education - Google for Developers, accessed April 30, 2025, https://developers.google.com/edu/python/dict-files
5. Built-in Types — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/library/stdtypes.html
6. 4. More Control Flow Tools — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/tutorial/controlflow.html
7. Python for Loops: The Pythonic Way, accessed April 30, 2025, https://realpython.com/python-for-loop/
8. Working with Lists — Python Beginners documentation - Read the Docs, accessed April 30, 2025, https://python-adv-web-apps.readthedocs.io/en/latest/lists.html
9. 5. Data Structures — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/tutorial/datastructures.html
10. Python Tuples - Tutorialspoint, accessed April 30, 2025, https://www.tutorialspoint.com/python/python_tuples.htm
11. Python while Loops: Repeating Tasks Conditionally, accessed April 30, 2025, https://realpython.com/python-while-loop/
12. Python For-Else and While-Else Clearly Explained with 4 Real-World Examples, accessed April 30, 2025, https://towardsdatascience.com/python-for-else-and-while-else-clearly-explained-with-4-real-world-examples-2b23ca6f6a51/
13. Python Nested Loops: Use, Work, Syntax, Example - WsCube Tech, accessed April 30, 2025, https://www.wscubetech.com/resources/python/nested-loops
14. Python While Loop | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/python-while-loop/
15. Python Control Flow and Loops (Learning Path) - Real Python, accessed April 30, 2025, https://realpython.com/learning-paths/python-control-flow-and-loops/
16. Python Do-While Loop: Ultimate How-To Guide - IOFLOOD.com, accessed April 30, 2025, https://ioflood.com/blog/python-do-while/
17. Python While Loop Tutorial - Server Academy, accessed April 30, 2025, https://serveracademy.com/blog/python-while-loop-tutorial/
18. How To Use Python Continue, Break and Pass Statements when Working with Loops, accessed April 30, 2025, https://www.digitalocean.com/community/tutorials/how-to-use-break-continue-and-pass-statements-when-working-with-loops-in-python-3
19. Python For Else | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/python-for-else/
20. Using Else Conditional Statement With For loop in Python - GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/using-else-conditional-statement-with-for-loop-in-python/

21. Why does python use 'else' after for and while loops? - Stack Overflow, accessed April 30, 2025, https://stackoverflow.com/questions/9979970/why-does-python-use-else-after-for-and-while-loops
22. Python Nested Loops | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/python-nested-loops/
23. 5.3 Nested loops - Introduction to Python Programming | OpenStax, accessed April 30, 2025, https://openstax.org/books/introduction-python-programming/pages/5-3-nested-loops
24. Nested for loop in Python: A Complete Guide | upGrad, accessed April 30, 2025, https://www.upgrad.com/tutorials/software-engineering/python-tutorial/nested-for-loop-in-python/
25. Introduction to Python Programming - Itasca Software Documentation Sets, accessed April 30, 2025, https://docs.itascacg.com/itasca900/common/docproject/source/manual/scripting/python/doc/basic_python.py.html
26. Lists (Arrays) - Easy Python Docs, accessed April 30, 2025, http://easypythondocs.com/lists.html
27. List Comprehensions - Practical Python, accessed April 30, 2025, https://practical.learnpython.dev/05_practical_applications/00_list_comprehensions/
28. List Comprehension in Python | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/python-list-comprehension/
29. When to Use a List Comprehension in Python, accessed April 30, 2025, https://realpython.com/list-comprehension-python/
30. Python List Comprehension: Tutorial With Examples, accessed April 30, 2025, https://python.land/deep-dives/list-comprehension
31. tuple — Python Reference (The Right Way) 0.1 documentation - Read the Docs, accessed April 30, 2025, https://python-reference.readthedocs.io/en/latest/docs/tuple/
32. The Python return Statement: Usage and Best Practices – Real Python, accessed April 30, 2025, https://realpython.com/python-return-statement/
33. dict — Python Reference (The Right Way) 0.1 documentation - Read the Docs, accessed April 30, 2025, https://python-reference.readthedocs.io/en/latest/docs/dict/
34. Dictionaries in Python – Real Python, accessed April 30, 2025, https://realpython.com/python-dicts/
35. Dictionaries — Python Beginners documentation - Read the Docs, accessed April 30, 2025, https://python-adv-web-apps.readthedocs.io/en/latest/dicts.html
36. Sets in Python – Real Python, accessed April 30, 2025, https://realpython.com/python-sets/
37. set — Python Reference (The Right Way) 0.1 documentation - Read the Docs, accessed April 30, 2025, https://python-reference.readthedocs.io/en/latest/docs/sets/

38. Sets in Python | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/sets-in-python/
39. Object-Oriented Programming (OOP) in Python – Real Python, accessed April 30, 2025, https://realpython.com/python3-object-oriented-programming/
40. What Is the With Statement in Python? | Built In, accessed April 30, 2025, https://builtin.com/software-engineering-perspectives/what-is-with-statement-python
41. Using the Python return Statement Effectively (Overview) (Video) - Real Python, accessed April 30, 2025, https://realpython.com/videos/python-return-overview/
42. Pass by Reference in Python: Background and Best Practices, accessed April 30, 2025, https://realpython.com/python-pass-by-reference/
43. Python import: Advanced Techniques and Tips, accessed April 30, 2025, https://realpython.com/python-import/
44. Create and Import modules in Python | GeeksforGeeks, accessed April 30, 2025, https://www.geeksforgeeks.org/create-and-import-modules-in-python/
45. Python Modules Tutorial: Importing, Writing, and Using Them | DataCamp, accessed April 30, 2025, https://www.datacamp.com/tutorial/modules-in-python
46. 6. Modules — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/tutorial/modules.html
47. Python Modules - w3schools.blog, accessed April 30, 2025, https://www.w3schools.blog/python-modules
48. 2.6 The math module - Introduction to Python Programming | OpenStax, accessed April 30, 2025, https://openstax.org/books/introduction-python-programming/pages/2-6-the-math-module
49. math — Mathematical functions — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/library/math.html
50. cmath — Mathematical functions for complex numbers — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/library/cmath.html
51. Generate pseudo-random numbers — Python 3.13.3 documentation, accessed April 30, 2025, https://docs.python.org/3/library/random.html
52. How to Create Custom Modules in Python (with examples) - wellsr.com, accessed April 30, 2025, https://wellsr.com/python/how-to-create-custom-modules-in-python/
53. Where to place custom module? - Python discussion forum, accessed April 30, 2025, https://discuss.python.org/t/where-to-place-custom-module/47097
54. How to Create and Import a Custom Module in Python - Stack Overflow, accessed April 30, 2025, https://stackoverflow.com/questions/37072773/how-to-create-and-import-a-custom-module-in-python
55. Reading and Writing Files in Python (Guide) – Real Python, accessed April 30, 2025, https://realpython.com/read-write-files-python/
56. Working With Files in Python, accessed April 30, 2025, https://realpython.com/working-with-files-in-python/

57. Real Python: Python Tutorials, accessed April 30, 2025, https://realpython.com/
58. where to start to learn python : r/learnpython - Reddit, accessed April 30, 2025, https://www.reddit.com/r/learnpython/comments/ulmfb8/where_to_start_to_learn_python/
59. Learning python with W3Schools : r/learnpython - Reddit, accessed April 30, 2025, https://www.reddit.com/r/learnpython/comments/19cee9d/learning_python_with_w3schools/
60. Python Basics, accessed April 30, 2025, https://realpython.com/tutorials/basics/